Cluster d'agents — AgentInsurFlow

Service FastAPI (Python, architecture hexagonale) qui évalue un dossier d'assurance auto : un graphe LangGraph déroule le plan de référence sur quatre agents (Pièces, Analyste, Rédacteur, Actuaire), écrit un brouillon, puis **attend la revue du gestionnaire** (`interrupt()`). Le LLM ne fait que rédiger : toute sortie est comparée à la référence déterministe et aux garde-fous, sinon repli.

Graphe

```
flowchart LR
    C[charger] --> P[pas]
    P -->|"agent à appeler"| P
    P -->|"proposer / revue humaine"| PR[proposer]
    PR -->|"EN_ANALYSE"| V["valider (interrupt)"]
    PR -->|"REVUE_HUMAINE"| F([fin])
    V -->|"POST .../revue"| E[enregistrer] --> F
```

Texte alternatif. `charger` lit le dossier et passe les observations au garde d'entrée ; `pas` boucle sur les agents tant que le plan en demande un ; `proposer` écrit le brouillon et le statut `EN_ANALYSE` (ou `REVUE_HUMAINE` : fin du graphe) ; `valider` met le graphe en pause jusqu'à `POST /evaluations/{numero}/revue`, puis `enregistrer` écrit la décision. `thread_id` = numéro de contrat.

API (jeton Entra, rôle `Gestionnaire`, audience `api://agentinsurflow-cluster` ou appId du cluster)

Route	Réponses
`POST /evaluations/{numero_contrat}`	`200 {statut, fiche, prime, score, message, synthese}` (revue humaine : `score` du barème ; message « Service temporairement indisponible » si le LLM est coupé ou en panne, sinon « Ce dossier nécessite une revue humaine. ») ; `404 CONTRAT_INTROUVABLE` (inexistant **ou** dossier d'un autre gestionnaire) ; `409 EVALUATION_EN_ATTENTE` / `DOSSIER_DECIDE` ; `502 OUTIL_INDISPONIBLE` ; `503 DELAI_DEPASSE`
`POST /evaluations/{numero_contrat}/revue {decision, validation, motif (1–500)}`	`200 {statut}` ; `404 CONTRAT_INTROUVABLE` (autre gestionnaire) ; `409 AUCUNE_REVUE_EN_ATTENTE` / `DOSSIER_DECIDE` ; `422 REQUETE_INVALIDE` (dont motif contenant e-mail, téléphone ou IBAN)
`GET /health` (sans jeton)	`200 {version, prompts, garde_fous, reglages}` ; `503` tant que LLM Guard n'est pas chargé

Contrat OpenAPI : [docs/openapi/agentic-cluster.json](), régénéré par `make openapi` (un test échoue s'il est périmé).

Routes publiques (gateway, T12) : `POST /v1/souscriptions/{numero_contrat}/evaluation` → `POST /evaluations/{numero_contrat}`, `POST /v1/souscriptions/{numero_contrat}/revue` → `POST`

`/evaluations/{numero_contrat}/revue`, `GET /health/cluster` → `GET /health`. Le document OpenAPI du cluster (`/cluster/openapi.json`, servi par la gateway) déclare déjà ces chemins publics, pour que la page Swagger unique appelle la gateway.

Qui peut appeler le cluster (R18). La gateway **relaie** le jeton du gestionnaire (audience du cluster), comme pour `/mcp` : elle ne l'échange pas. « N'accepte que la gateway » (D105) est garanti par l'**entrée interne** de la Container App (aucune adresse joignable hors de l'environnement) et par la **revalidation du jeton** par le cluster (signature, émetteur, audience, rôle `Gestionnaire`), et non par `azp = gateway`. Le cluster échange ensuite ce jeton pour chaque agent (On-Behalf-Of en deux sauts : cluster, puis agent vers le MCP), d'où un `azp` propre à chaque agent côté MCP.

Débit. Les routes publiques sont limitées à 10 requêtes/min (`RateLimit__Cluster__PermitLimit`) par `oid` et `azp` du jeton, côté gateway (`gateway/README.md`) : 429 au-delà.

Erreurs : `application/problem+json` avec un `code` stable. Le message de `REVUE_HUMAINE` est neutre : jamais la cause ni le nom d'un détecteur.

Concurrence et reprise

- Un `asyncio.Lock` par numéro de contrat sérialise `evaluer` et `reviser` : il ne vaut que pour **une réplique** (`maxReplicas: 1` en T12) ; plusieurs répliques demanderaient un verrou en base. **Limite (R17)** : pendant un déploiement, l'ancienne et la nouvelle révision peuvent coexister brièvement (même avec `maxReplicas: 1`) ; le verrou par dossier n'est pas garanti pendant cette fenêtre.
- Un dossier `ACCEPTE` ou `REFUSE` ne se réévalue pas : `409 DOSSIER_DECIDE`. L'administration le fige aussi (`409 STATUT_FIGE` sur `PUT …/statut` et `…/brouillon`, code d'outil `STATUT_FIGE` côté MCP) : le cluster le traduit en `409 DOSSIER_DECIDE`, la décision humaine n'est pas réversible.
- **Décider un dossier en `REVUE_HUMAINE` (7a).** Il n'y a pas de pause : le graphe est terminé et `/revue` répond `409 AUCUNE_REVUE_EN_ATTENTE`. Le gestionnaire décide lui-même par `PUT /api/contrats/{numero}/statut` (`{"statut": "ACCEPTE"|"REFUSE", "validation": "VALIDEE"|"CORRIGEE"}`) sur la gateway, avec son propre jeton ; la fiche provisoire est le brouillon à valider. Une réévaluation (`POST /evaluations/{n}`) reste possible tant que le dossier n'est pas décidé.
- Si l'écriture de la décision échoue (`502`), la revue reste posée : un second `/revue` rejoue `enregistrer` (la décision la plus récente l'emporte) ; `POST /evaluations/{n}` répond `409 EVALUATION_EN_ATTENTE` entre-temps.
- **Propriétaire du dossier (BOLA).** L'`oid` du gestionnaire est posé dans l'état du graphe au premier `charger` ; un autre `oid` reçoit `404 CONTRAT_INTROUVABLE` aux deux routes, avant tout `409`, sans toucher au point de reprise. `tentative` croît de 1 à chaque évaluation (lue dans le point de reprise) : les tours et rapports d'une réévaluation ne sont pas ignorés par l'administration.
- **Points de reprise (D106).** Stockés par la gateway dans underwriting-admin (`/api/points-de-reprise`, rôle `PointsDeReprise`), plus d'accès PostgreSQL depuis le cluster (`infrastructure/langgraph/checkpoint_http.py`). Une erreur de jeton ou HTTP remonte : jamais de point vide ni de repli.
- Les observations du dossier ne sont jamais dans l'état du graphe ni dans un point de reprise : elles passent par le contexte d'exécution (en mémoire).

Réglage des agents (`agents.toml`)

`agentic-cluster/agents.toml` règle chacun des quatre sous-agents (`pieces`, `analyste`, `redacteur`, `actuaire` ; la Saga n'a pas de LLM) : `modele` (déploiement Foundry), `temperature` ∈ [0, 1], `max_tokens` entier dans [1, 16 000], `delai_s` > 0, `prompt` (fichier de `infrastructure/llm/prompts/`). Aucune autre section ni clé ; la somme des `delai_s` ne dépasse pas `INSURFLOW_BUDGET_S` (25 s).

- **Modifier par PR** : le fichier est copié dans l'image et lu au démarrage ; pas de variable, pas de réglage à chaud.
- **Validé au démarrage** : un réglage invalide empêche le cluster de démarrer (message : agent et clé).
- **Vérifié au déploiement** (`deploy.yml`, avant toute mise à jour) : le fichier du SHA déployé est lu et chaque `modele` doit exister comme déploiement de la ressource Foundry (`vars.FOUNDRY_ENDPOINT`, lecture seule) ; sinon le déploiement est refusé avec le nom manquant. Un rollback vérifie donc l'ancien réglage.
- Les modèles ne sont plus dans un `llm.env` : le cluster n'a ni `agentic-cluster/llm.env` ni `AZURE_OPENAI_DEPLOYMENT` (rag garde `rag/llm.env`, `Kimi-K2.5`). Réglage exposé par `GET /health` (`reglages`) et par `audit.agent` / `invoke_agent`.

Configuration (variables d'environnement, aucune valeur dans le dépôt)

- Entra et gateway : `AZURE_TENANT_ID`, `GATEWAY_URL`, `MCP_API_CLIENT_ID`, `LLM_AUDIENCE`, `INSURFLOW_CLUSTER_CLIENT_ID`, `INSURFLOW_AGENT_<AGENT>_CLIENT_ID`, `INSURFLOW_CLIENT_SECRET` (local) ou `AZURE_UAMI_CLIENT_ID` (Azure).
- Points de reprise : par la gateway si `TELEMETRIE_AUDIENCE` (audience d'underwriting-admin), `AZURE_TENANT_ID` et `INSURFLOW_CLUSTER_CLIENT_ID` sont posés ; sinon mémoire avec un avertissement, sauf `APP_ENV` = `staging` ou `production` où le démarrage échoue.
- Garde globale : `INSURFLOW_BUDGET_S` (25 s) et `INSURFLOW_RESERVE_S` (2 s) ; budget d'un agent = `min(délai de l'agent, 25 − écoulé − 2)`, sous 0,3 s : revue humaine (`duree`). Filet : `asyncio.timeout(budget + réserve)` autour du graphe ; dépassé pendant la boucle des agents ⇒ revue humaine `duree` (3 s au plus pour conclure, 30 s au total) ; dépassé ailleurs (lecture du contrat, écriture du brouillon) ⇒ `503 DELAI_DEPASSE` (rien de cohérent à proposer, l'évaluation se relance).
- Appel d'outil MCP (jeton OBO compris) : borné à 22 s (`DELAI_APPEL_S`, au-dessus des 20 s du calcul côté MCP) ; dépassé ⇒ `MCP_INDISPONIBLE`, rapport en repli `erreur` tracé (revue humaine) au lieu d'une garde globale épuisée sans rapport.
- Observabilité : `OTEL_SERVICE_NAME=agentic-cluster`, `OTEL_EXPORTER_OTLP_*` ; en Azure, `TELEMETRIE_AUDIENCE` (`api://<appId underwriting>`) : export par la gateway (`/v1/traces`) avec un jeton d'application du cluster (rôle `Telemetrie`), comme le serveur MCP ; local : export direct, sans jeton. Un enregistrement `audit.agent` (journal `insurflow.audit`) et un span `invoke_agent` par agent ; ni jeton, ni nom, ni prompt, ni observations (empreintes SHA-256 seulement).

Lancer

```
make test                                    # TU
uv run pytest agentic-cluster/tests --no-cov  # TU + TI (hors modèles réels, `-m lent`)
docker compose build agentic-cluster        # image : modèles LLM Guard cuits (≈ 2,5 Go de
modèles)
```

Pour aller plus loin

- État du contrat personnel K1→K11, avenants, mesures des garde-fous et signaux du sprint 5 : docs/contrat-personnel.md.
- Démo de l'atelier 4 : docs/demo-atelier-4.md.